

HZ-RK3562-SP_EVM-SDK 使用说明



主题	SDK 使用说明
文档号	1.0
创建时间	2025-04-25
最后修改	2025-04-25
版本号	1.0
文件名	HZ-RK3562-SP_EVM-SDK 使用说明 V1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-04-25	邓鑫怡	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 编译准备工作	6
1.1 开发环境搭建	6
1.1.1 配置开发环境	6
1.1.2 安装软件依赖包	6
1.2 SDK 安装	7
1.2.1 拷贝 SDK 到虚拟机	7
1.2.2 解压 SDK	7
1.3 SDK 目录介绍	8
第 2 章 SDK 构建方法	9
2.1 配置开发板参数	9
2.1.1 导入开发板配置	9
2.1.2 SDK 定制化配置	9
2.1.3 内核配置	11
2.2 整体编译	12
2.3 单独编译	13
2.3.1 编译 UBoot	13

2.3.2 编译 kernel	14
2.3.3 编译 rootfs	16
2.4 清除编译	17

第 1 章 编译准备工作

1.1 开发环境搭建

1.1.1 配置开发环境

在进行相关编译工作时，推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

- 硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。
- 软件要求：Ubuntu 22.04 或更高版本系统。

具体配置虚拟机编译环境时，请参考《HZ-RK3562-SP_EVM-Linux 开发环境搭建》（路径为“4、文档教程/开发板使用手册”）下载配置虚拟机编译环境；如图 1.1-1。



图 1.1-1 文档位置

1.1.2 安装软件依赖包

可以通过以下步骤安装编译 SDK 需要的库和工具。使用如下 apt-get 命令安装后续操作所需的库和工具：

```
hzhy@ubuntu:~$ sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \ liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \ diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \ cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \ libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \ libmpc-dev bc python-is-python3 python2
```

说明：

安装过程中，请保证 Ubuntu 可正常访问互联网，若提示“*** is already the newest version ***”表示该软件已安装，请忽略。

1.2 SDK 安装

1.2.1 拷贝 SDK 到虚拟机

使用 SFTP 软件将资源包中的“HZ-RK3562-LINUX_5.10_SDK_V1.0.tar.gz 压缩包（路径：3、镜像及源码/HZ-RK3562-SP_EVM_Linux_SDK/）”拖动到虚拟机的/home/hzhy 工作目录下，详情参考文档《HZ-RK3562-SP_EVM-调试工具安装手册》。如图 1.2-1；



图 1.2-1 文档位置

1.2.2 解压 SDK

- 1) 使用 tar 命令，解压 SDK 压缩包到/home/JS/project 目录下。

```
hzhy@ubuntu:~$ tar -xvf HZ-RK3562-LINUX_5.10_SDK_V1.0.tar.gz
```

- 2) 执行该命令后，工作目录中将生成“HZ-RK3562-LINUX_5.10_SDK_V1.0”文件夹，可前往该目录查看 SDK 内容，如图 1.2-2。

```
hzhy@ubuntu:~$ ls
HZ-RK3562-LINUX_5.10_SDK_V1.0
hzhy@ubuntu:~$ cd HZ-RK3562-LINUX_5.10_SDK_V1.0
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ls
app          common      device      hal          Makefile    README.md   rockdev     u-boot      yocto
buildroot    Copyright_Statement.md docs         kernel       output      rkbin       rtos        Ubuntu20.04
build.sh     debian      external    kernel-5.10  prebuilts  rkflash.sh  tools       Ubuntu22.04
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$
```

图 1.2-2 压缩包目录

1.3 SDK 目录介绍

SDK 目录中的文件说明如表 1.3-1。

表 1.3-1 Linux SDK 源码目录结构

目录	说明
buildroot	存放基于 Buildroot 开发的根文件系统，用于构建嵌入式 Linux 系统。
build.sh	作为 LinuxSDK 编译脚本，用于指挥完成整个 SDK 项目的编译工作。
device	包含芯片板级配置、编译与打包固件的脚本及预备文件，助力芯片相关固件处理。
external	存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
kernel	用于存放 Linux 内核源码，是系统运行核心部分的代码所在处。
Makefile	为 LinuxSDK 的顶层 Makefile，起着统筹编译规则的作用。
prebuilts	存放交叉编译工具链，便于编译出适用于目标芯片的代码。
README.md	提供项目相关说明的 README 文件。
rkbina	存放 Rockchip 相关二进制和工具，服务于芯片相关操作。
u-boot	存放 U-Boot 源码，其作为引导加载器用于初始化硬件并加载主操作系统。
app	存放上层应用，主要是 qcamera/qfm/qplayer/qsetting 等一些应用程序。
debian	存有基于 Debian 12 开发的根文件系统，为系统运行提供基础环境。
docs	存放开发指导文件、平台支持列表、工具使用文档、Linux 开发指南等。
tools	存放 Linux 和 Windows 操作系统下常用工具，方便开发及相关操作。
yocto	存放基于 Yocto 5.0 开发的根文件系统，用于构建定制化的系统环境。

第 2 章 SDK 构建方法

2.1 配置开发板参数

2.1.1 导入开发板配置

解压后的 SDK 内置了 HZ-RK3562-SP_EVM 板卡的配置文件，编译 SDK 前需要选择对应文件并进行配置，方法如下。如图 2.1-1；

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh lunch //选择配置文件，选择 4 号文件
```

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh lunch
Log colors: message notice warning error fatal
Log saved at /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/sessions/2025-05-08_16-16-15
Pick a defconfig:
1. rockchip_defconfig
2. hzhy_rk3562_btb_defconfig
3. hzhy_rk3562_mini_defconfig
4. hzhy_rk3562_sp_defconfig
which would you like? [1]: 4
Switching to defconfig: /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/device/rockchip/.chip/hzhy_rk3562_sp_defconfig
#
# configuration written to /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/.config
#
Using last kernel version(5.10)
```

图 2.1-1 选择配置文件

2.1.2 SDK 定制化配置

SDK 可通过 make menuconfig 进行相关配置，通过 make menuconfig 配置界面提供了一个基于文本的用户界面来选择和配置各种选项。

配置完成后，使用 make savedefconfig 命令保存这些配置，将临时配置改为持久化配置。如图 2.1-2。

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ make menuconfig //配置文件
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ make savedefconfig //保存配置
```

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ make menuconfig
```

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ make savedefconfig
```

图 2.1-2 配置文件并保存配置

通过以上 config，可选择不同 Rootfs、Loader、Kernel 等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。目前可配组建如图 2.1-3。

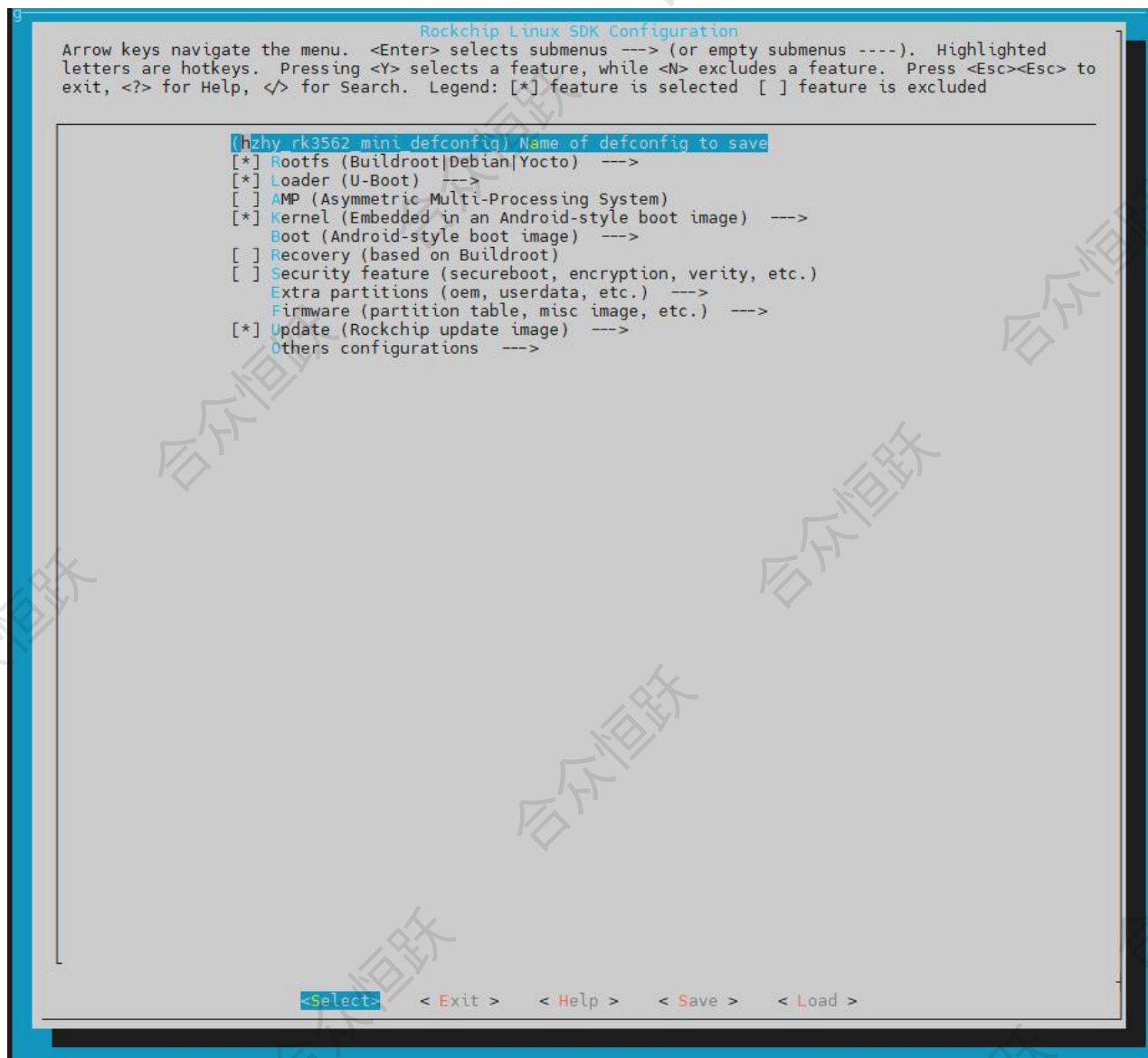


图 2.1-3 配置界面

配置相关内容说明：

- **Rootfs**：这里的 Rootfs 代表“根文件系统”，在这里可选择 Buildroot、Yocto、Debian 等不同的根文件系统配置。
- **Loader (u-boot)**：这是引导加载器的配置，通常是 u-boot，用于初始化硬件并加载主操作系统。
- **AMP**：多核异构系统开发配置选项,适用于电力、工控等行业。
- **Kernel**：这里配置内核选项，定制适合自己的硬件和应用需求的 Linux 内核。
- **Boot**：这里配置 Boot 分区支持格式。
- **Recovery (based on buildroot)**：这是基于 buildroot 的 recovery 环境的配置，用于系统恢复和升

级。

- **Security:** 安全功能开启, 包含 Secureboot 开启方式、Optee 存储方式、烧写 Key 等。
- **Extra partitions:** 用于配置额外的分区。
- **Firmware:** 在这里配置固件相关选项。
- **Update (Rockchip update image):** 用于配置 Rockchip 完整固件的选项。
- **Others configurations:** 其他额外的配置选项。

具体操作步骤:

- 可通过键盘的方向键选中对应菜单栏。在<Select>被选中的情况下, 可按“Enter”键进入子菜单。菜单选项中蓝色高亮的字母代表此菜单选项的快捷键, 可在键盘上按下对应的字母快速选中对应的菜单选项。每个菜单选项前的括号内容表示当前菜单选项的配置状态。
- 选中对应的菜单选项后, 按下 Y 键, 会将相应的选项选中, 同时菜单选项前面变为< * >。按下 N 键, 不会将相应的选项选中。如需搜索, 可按下/键打开搜索框, 然后输入要搜索的内容。
- 配置完毕后, 选中<Save>, 按 Enter 键保存配置选项。选中<ok>, 按 Enter 键, 然后双击 Esc 键退出。更多详情请参考瑞芯微 SDK 开发手册。

2.1.3 内核配置

Kernel 配置可通过执行“./build.sh kernel-config”命令来进行。执行该命令后, 会出现一个基于文本的用户界面, 在这个配置界面中, 能够选择和配置各种选项。

借助该界面, 可以选择不同的配置开展各种定制化编译, 进而灵活地对系统组件进行选择与配置, 以此满足特定的需求。当前可配置的组件情况可参照图 2.1 - 4 查看。

```
hzy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh kernel-config
```

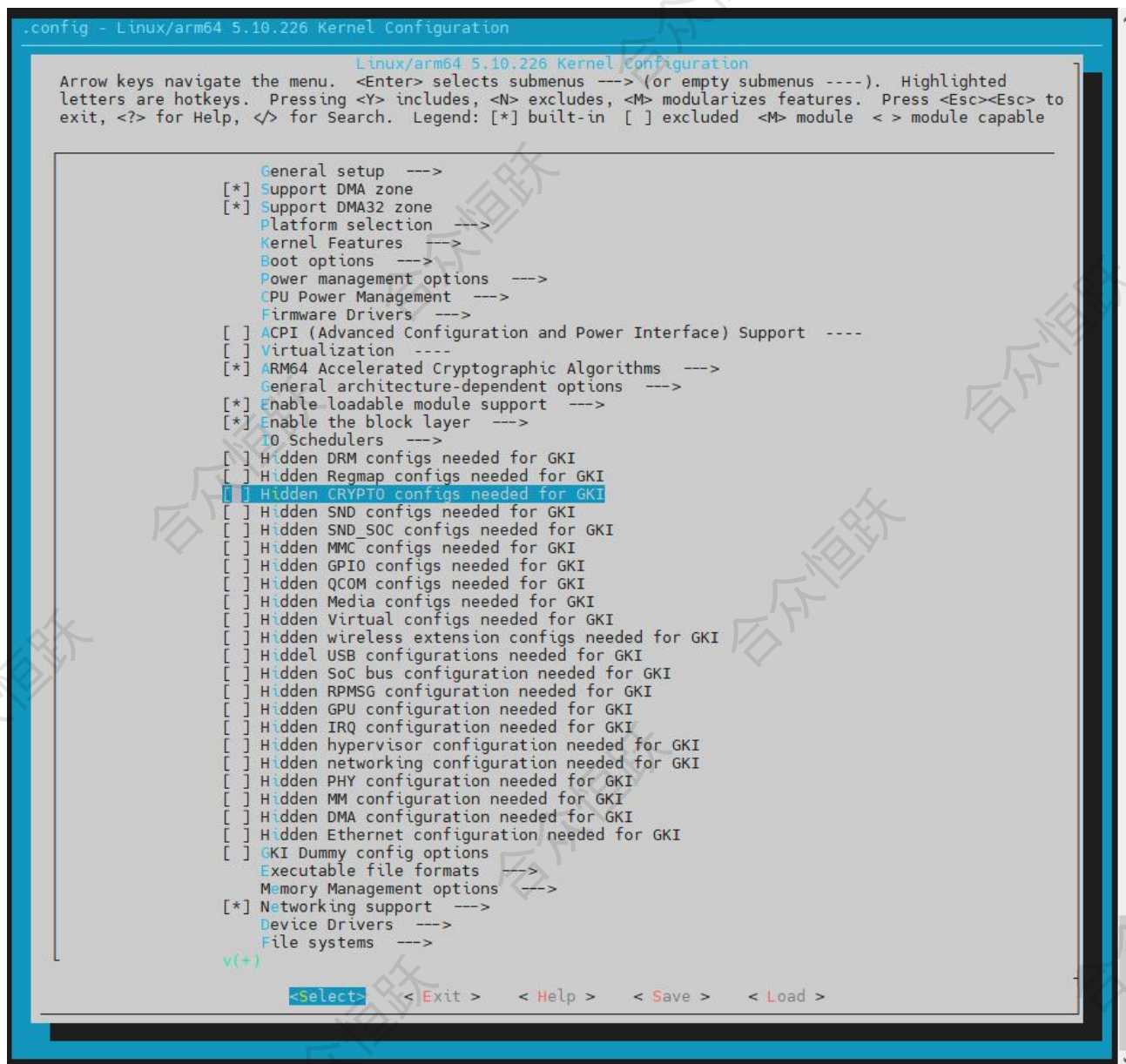


图 2.1-4 配置界面

2.2 整体编译

在 SDK 根目录下执行以下命令整体编译，部分输出如图 2.2-1 所示：

```
hzhhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh
```



```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh
Log colors: message notice warning error fatal

Log saved at /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/sessions/2025-05-08_15-58-01
WARN: /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/device/rockchip/common/configs is updated
#
# configuration written to /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/.config
#
Using last kernel version(5.10)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_hzhy_rk3562_sp
RK_CHIP=rk3562
RK_CHIP_FAMILY=rk3562
RK_CHIP_HAS_GPU=y
RK_DEFCONFIG=hzhy_rk3562_sp_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=hzhy_rk3562_sp_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/hzhy_rk3562_sp.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_RAW=5.10
RK_LOADER=y
RK_MISC=y

Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.29*****
Generating new image, please wait...
Writing head info ...
Writing boot file ...
Writing firmware ...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build updateimg succeeded.
Images under /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware/ are ready!
Running mk-firmware.sh - build firmware succeeded.
Running 99-all.sh - build all succeeded.
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$
    
```

图 2.2-1 整体编译

整体编译成功后，将在 `output/firmware` 文件夹下生成新的编译工程文件夹，可以找到对应目录其中的镜像文件进行烧录，如图 2.2-2。

```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ cd output/firmware/
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware$ ls
boot.img  MiniLoaderAll.bin  misc.img  oem.img  parameter.txt  rootfs.img  uboot.img  update.img  userdata.img
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware$
    
```

图 2.2-2 镜像文件目录

注意：update.img 为已打包的完整镜像文件，适用于整包烧写；其他文件为分区镜像，适用于分步骤烧写。

2.3 单独编译

2.3.1 编译 UBoot

在 SDK 根目录下执行以下命令编译 U-Boot，部分输出如图 2.3-1 所示：

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh uboot
```

```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh uboot
Log colors: message notice warning error fatal
Log saved at /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/sessions/2025-05-08_16-05-42
Using last kernel version(5.10)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_hzhy_rk3562_sp
RK_CHIP=rk3562
RK_CHIP_FAMILY=rk3562
RK_CHIP_HAS_GPU=y
RK_DEFCONFIG=hzhy_rk3562_sp_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=hzhy_rk3562_sp_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/hzhy_rk3562_sp.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_RAW=5.10
RK_LOADER=y
RK_MISC=y

*****boot_merger ver 1.35*****
Info:Pack loader ok.
creating new idblock from loader ...
idblock binary saving at idblock.img
pack loader(SPL) okay! Input: /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/rkbin/RKBOOT/RK3562MINIALL.ini

/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/u-boot
pack loader with new: spl/u-boot-spl.bin

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3562_spl_loader.v1.07.106.bin (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/rkbin/RKTRUST/RK3562TRUST.ini

Platform RK3562 is build OK, with new .config(make hzhy_rk3562_sp_defconfig -j64)
/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64
-none-linux-gnu/bin/aarch64-none-linux-gnu-
Thu May 8 16:05:55 CST 2025
+ cd ..
Running 70-loader.sh - build_uboot succeeded.
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$
    
```

图 2.3-1 编译 uboot

编译成功后，系统将在 u-boot 目录下生成新的 uboot.img 文件，该文件为 U-Boot 烧写镜像，如图

2.3-2 所示。

```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ cd u-boot/
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/u-boot$ ls
apt          common      dts         lib          README      rk3562_spl_loader_v1.07.106.bin  tools      u-boot.img
arch         config.mk  env         licenses     rk3562_spl_loader_v1.07.106.bin  scripts    u-boot.lds
bl31_0x00400000.bin  configs   examples   MAINTAINERS  snapshot.commit  spl        u-boot.map
bl31_0xfe481000.bin  defconfig fit         Makefile     spl            u-boot.bin  u-boot-nodtb.bin
bl31_0xfe490000.bin  disk      fs          make.sh      System.map    u-boot.cfg  u-boot.srec
bl31.elf         doc        include    net          tee.bin       u-boot.cfg.configs  u-boot.sym
board           Documentation  Kbuild     post         test          u-boot.dtb  usb_update.txt
cmd            drivers     Kconfig    PREUPLOAD.cfg  test          u-boot-dtb.bin
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/u-boot$
    
```

图 2.3-2 uboot.img 文件目录

2.3.2 编译 kernel

在 SDK 根目录下执行以下命令编译 kernel，部分输出如图 2.3-3 所示：

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh kernel
```

```
JS@ubuntu:~/project/01_HZ-EVM-RK3562SP_LINUX5.10_V1.3.0_SDK$ ./build.sh kernel

##### Rockchip Linux SDK #####
Manifest: rk3562_linux_release_v1.3.0_20241220.xml
Log colors: message notice warning error fatal
Log saved at /home/JS/project/01_HZ-EVM-RK3562SP_LINUX5.10_V1.3.0_SDK/output/sessions/2025-04-25_16-52-54
Using last kernel version(5.10)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/JS/project/01_HZ-EVM-RK3562SP_LINUX5.10_V1.3.0_SDK/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3562
RK_CHIP=rk3562
RK_CHIP_FAMILY=rk3562
RK_CHIP_HAS_GPU=y
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=HZ-EVM-RK3562SP_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/HZ-EVM-RK3562SP.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_RAW=5.10
RK_LOADER=y
RK_MISC=y
RK_MISC_BLANK=y
RK_NETWORK_CHECK=y
RK_OVERLAY=y
RK_OWNER=JS
RK_OWNER_UID=1001
RK_PARAMETER=parameter-buildroot-fit.txt
RK_RECOVERY=y
RK_RECOVERY_CFG=rockchip_rk3562_recovery
RK_RECOVERY_CPIO_GZ=y

OS: Linux
Load Address: 0xfffffffff01
Entry Point: 0xfffffffff01
Hash algo: sha256
Hash value: fdd25cde0fbff208def703ec13315d1e5cf7c9f8da22843ec4ff3cdd9cded165
Image 2 (resource)
Description: unavailable
Created: Thu May 8 16:07:42 2025
Type: Multi-File Image
Compression: uncompressed
Data Size: 1027072 Bytes = 1003.00 KiB = 0.98 MiB
Hash algo: sha256
Hash value: d3a95bd38be441980230540516f000a4e0e1b8c7e7bacbd5946dc2fb3c3042e8
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
+ ln -rsf kernel/boot.img /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm64/boot/dts/rockchip/hzhy_rk3562_sp.dts
Running 10-kernel.sh - build kernel succeeded.
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$
```

图 2.3-3 编译 kernel

编译成功后，系统将在 kernel 目录下生成新的 boot.img 文件，该文件为 boot 烧写镜像，如图 2.3-4 所示。


```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ cd kernel
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel$ ls
android          build.config.gki-debug.x86_64    include          net
arch             build.config.gki_kasan          init            OWNERS
block            build.config.gki_kasan.aarch64  io_uring        README
boot.img         build.config.gki_kasan.x86_64   ipc             README.md
boot.its         build.config.gki_kprobes        Kbuild          resource.img
build.config.aarch64  build.config.gki_kprobes.aarch64  Kconfig        samples
build.config.allmodconfig  build.config.gki_kprobes.x86_64  kernel          scripts
build.config.allmodconfig.aarch64  build.config.hikey960          lib             security
build.config.allmodconfig.arm      build.config.khwasan           LICENSES        sound
build.config.amlogic              build.config.rockchip          logo.bmp        System.map
build.config.arm                  build.config.x86_64            logo_kernel.bmp tools
build.config.common               certs                          MAINTAINERS     usr
build.config.db845c               COPYING                       Makefile         virt
build.config.gki                  CREDITS                      mm               vmlinux
build.config.gki.aarch64          crypto                        modules.builtin  vmlinux.o
build.config.gki.aarch64.fips140  Documentation                modules.builtin.modinfo  vmlinux.symvers
build.config.gki.aarch64.fips140_eval_testing  drivers                      modules-only.symvers    zboot.img
build.config.gki-debug.aarch64    fs                            modules.order
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/kernel$
    
```

图 2.3-4 boot.img 文件目录

2.3.3 编译 rootfs

在 SDK 根目录下执行以下命令编译 rootfs，部分输出如图 2.3-5 所示：

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh rootfs
```

```

hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh rootfs
Log colors: message notice warning error fatal
Log saved at /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/sessions/2025-05-08_16-09-33
Using last kernel version(5.10)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_hzhy_rk3562_sp
RK_CHIP=rk3562
RK_CHIP_FAMILY=rk3562
RK_CHIP_HAS_GPU=y
RK_DEFCONFIG=hzhy_rk3562_sp_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=hzhy_rk3562_sp_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/hzhy_rk3562_sp.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
RK_KERNEL_VERSION=5.10
RK_KERNEL_VERSION_RAW=5.10
RK_LOADER=y
RK_MISC=y
RK_MISC_BLANK=y
RK_NETWORK_CHECK=y
RK_OVERLAY=y
RK_OWNER=hzhy
RK_OWNER_UID=1012
RK_PARAMETER=hzhy_rk3562_sp_parameter.txt
RK_ROOTFS=y

rockchip_hzhy_rk3562_sp/target/usr/lib/modules/stU3Rr7o' [elf64-littleaarch64]
Running 95-extra-parts.sh for /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/buildroot/output/rockchip_hzhy_rk3562_sp/target (buildroot init=busybox) ...
Preparing extra partitions ...
Running 97-ldcache.sh for /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/buildroot/output/rockchip_hzhy_rk3562_sp/target (buildroot init=busybox) ...
Keep original /etc/ld.so.cache
Running 99-owner.sh for /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/buildroot/output/rockchip_hzhy_rk3562_sp/target (buildroot init=busybox) ...
Running 30-rootfs.sh - build_buildroot /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/buildroot/images succeeded.
Running 30-rootfs.sh - build rootfs rootfs succeeded.
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$
    
```

图 2.3-5 编译 rootfs

编译成功后，系统将在 buildroot/output/rockchip_hzhy_rk3562_sp/images/目录下生成新的 rootfs.ext4 文件，该文件为 rootfs 烧写镜像，如图 2.3-6 所示。


```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ cd buildroot/output/rockchip_hzhy_rk3562_sp/images/  
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0/buildroot/output/rockchip_hzhy_rk3562_sp/images$ ls  
rootfs.cpio  rootfs.cpio.gz  rootfs.ext2  rootfs.ext4  rootfs.squashfs  rootfs.tar
```

图 2.3-6 rootfs.ext4 文件目录

2.4 清除编译

`cleanall` 命令会删除编译的中间文件和编译生成文件，执行该命令后需要全部重新编译才能得到生成文件，如图 2 4-1 清除所有文件。发布的 SDK 源码包已经配置好各项参数，非必要不建议完全重新编译。

注意：完全重新编译会非常耗时（i5-12400，6 核 12 线程编译一次大概两小时）。

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh cleanall
```

```
hzhy@ubuntu:~/HZ-RK3562-LINUX_5.10_SDK_V1.0$ ./build.sh cleanall  
Log colors: message notice warning error fatal  
Log saved at /home/LRX/project/gpio_check/HZ_RK3562_SDK/HZ-RK3562-LINUX_5.10_SDK_V1.0/output/sessions/2025-05-08_16-15-17
```

图 2.4-1 清除所有文件